

Programmation – TD – Arbres binaires

Méthodes de la classe BinaryTree

Ces méthodes utilisent le type énuméré :

— enum NodeType {SENTINEL, LEAF, SIMPLE_LEFT, SIMPLE_RIGHT, DOUBLE}

Constructeurs de la classe BinaryTree<T>

BinaryTree()	instancier A et créer un nœud vide (butoir) en tant que la racine de l'arbre
--------------	--

Méthodes de la classe BinaryTree<T>

Iterator<T> iterator()	<i>résultat</i> → créer un itérateur sur l'arbre binaire, le nœud courant de l'itérateur est positionné sur la racine de l'arbre
boolean isEmpty()	<i>résultat</i> → vérifier si l'arbre est vide

Constructeurs de la classe Iterator<T>

Il n'existe pas de constructeurs publics de la classe Iterator.

Méthodes de la classe Iterator<T>

void goLeft()	<i>prérequis</i> → le nœud courant n'est pas un butoir <i>effet</i> → se positionner sur le fils gauche du nœud courant
void goRight()	<i>prérequis</i> → le nœud courant n'est pas un butoir <i>effet</i> → se positionner sur le fils droit du nœud courant
void goUp()	<i>prérequis</i> → le nœud courant n'est pas la racine <i>effet</i> → se positionner sur le père du nœud courant
void goRoot()	<i>effet</i> → se positionner sur la racine de l'arbre
boolean isEmpty()	<i>résultat</i> → vérifier si le nœud courant est un butoir
NodeType nodeType()	<i>résultat</i> → le genre du nœud courant
void remove()	<i>prérequis</i> → le nœud courant n'est pas un nœud double <i>effet</i> → retirer le nœud courant
void clear()	<i>effet</i> → vider le sous-arbre référencé par le nœud courant, qui devient butoir
T getValue()	<i>résultat</i> → la valeur du nœud courant
void addValue(T v)	<i>prérequis</i> → le nœud courant est un butoir <i>effet</i> → créer un nouveau nœud de valeur v à cet endroit
void setValue(T v)	<i>effet</i> → affecter la valeur v au nœud courant
void switchValue(int i)	<i>prérequis</i> → $i \geq 0$ et racine est père du nœud courant d'ordre $\geq i$ <i>effet</i> → échanger les valeurs associées au nœud courant et à son père d'ordre i (le nœud courant reste inchangé)

Arbres Binaires

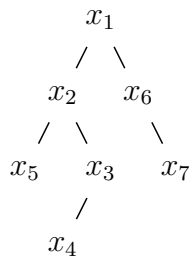
1. Vue abstraite d'un arbre binaire

Arbre orienté : graphe orienté $G = (X, \Gamma)$ t.q.

- $\exists x_1 \in X$ racine de G (x_1 point d'entrée unique) et $\forall x \in X \exists$ chemin $[x_1 * x]$
- $\forall x \neq x_1 |\Gamma^{-1}| == 1$ (x a un seul père) (donc pas de circuits)

Arbre binaire : arbre orienté t.q.,

- $\forall x \in X - |\Gamma(x)| \leq 2$
- les (éventuels) successeurs de x sont ordonnés (fils gauche, fils droit)



Terminologie :

- X = nœuds de l'arbre
- $\forall x \in X : [x_1 * x]$ est unique et s'appelle la branche de x
- x_1 s'appelle la racine de l'arbre
- tout nœud x est de l'un des 4 genres :
 - double si $|\Gamma(x)| == 2$ (ici x_1 et x_2)
 - feuille si $|\Gamma(x)| == 0$ (x_4 , x_5 et x_7)
 - simple à gauche si $|\Gamma(x)| == 1$ et pas de fils droit (x_3)
 - simple à droite si $|\Gamma(x)| == 1$ et pas de fils gauche (x_6)
- la profondeur, ou le niveau, d'un nœud x est la longueur du chemin $[x_1 * x]$
- la hauteur de l'arbre vaut $\max_{x \text{ feuille}} (\text{profondeur}(x))$
- la hauteur d'un nœud x est la hauteur du sous-arbre de racine x

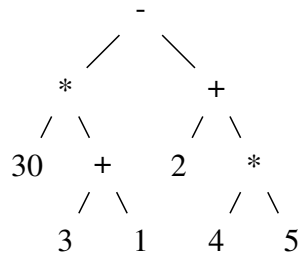
Le constructeur BinaryTree() crée un arbre vide.

Comme pour les listes, on a intérêt à "banaliser" les cas particuliers (arbre vide, absence d'un ou des 2 fils). À cette fin, on introduit les butoirs qui remplacent chaque fils absent. Les butoirs n'ont pas de fils.

Pour réaliser les déplacements et les modifications, les méthodes de la classe Itérateur gèrent un attribut currentNode.

2. Parcours canoniques

- parcours préfixe : racine puis, récursivement, sous-arbre gauche puis sous-arbre droit
- parcours postfixe : sous-arbre gauche, droit puis racine
- parcours infixé (symétrique) : sous-arbre gauche, racine, sous-arbre droit

Exercice 1.

Donner les 3 énumérations.

Exercice 2.

Écrire trois procédures récursives

```

static void preOrderTraversal(BinaryTree<MyInteger> tree)
static void inOrderTraversal(BinaryTree<MyInteger> tree)
static void postOrderTraversal(BinaryTree<MyInteger> tree)

```

qui affichent, selon l'ordre canonique concerné, les valeurs (entières) contenues dans l'arbre binaire référencé par tree.

Exercice 3.

Opérations arithmétiques, à opérandes entiers chiffres, avec, selon l'ordre décroissant des priorités :

- unaire
- * binaire
- +, - binaires

En cas d'égalité des priorités, l'évaluation est effectuée de gauche à droite. On peut modifier l'ordre d'évaluation à l'aide de parenthèses. Par convention, le - unaire est simple à droite et les feuilles ont une valeur chiffre.

- a) donner l'arbre d'évaluation associé à l'expression $- 9 - (5 - 4) + 8 * - (1 - 2)$
- b) écrire la fonction :

```

1  /**
2   * @param tree
3   *       arbre à évaluer
4   * @pre !tree.isEmpty()
5   * @pre les feuilles ont pour valeur un caractere chiffre
6   * @return valeur de l'expression représentée dans l'arbre tree
7   */
8  static int evaluate(BinaryTree<Character> tree)

```

Exercice 4. Gestion d'un stock de pièces détachées

Chaque pièce est caractérisée par un entier positif. Le stock disponible chez un fournisseur est représenté dans un arbre `BinaryTree<Part>`, avec

```
1 public class Part
2 {
3     /**
4      * numéro de la pièce
5      */
6     public int number;
7     /**
8      * nombre (strictement positif) d'exemplaires disponibles
9      */
10    public int quantity;
11    ...
12 }
```

```
1 public class Supplier extends BinaryTree<Part>
2 {
3     /**
4      * nombre maximal de pièces
5      */
6     private static int MAX_SIZE = 1000;
7     /**
8      * niveau de remplissage de this.table
9      */
10    private int n;
11    /**
12     * table auxiliaire pour la méthode arrange
13     */
14    private Part[] table;
15
16    public Supplier()
17    {
18        super();
19        n = 0;
20        table = new Part[MAX_SIZE];
21    }
22    ...
23 }
```

L'arbre d'un fournisseur est organisé pour une recherche symétrique sur le champ *number* des pièces. Écrire les méthodes d'instance suivantes de la classe `Supplier`.

```
1 /**
2  * Afficher, par numeros décroissants, les pièces disponibles dans
3  * this en moins de k exemplaires.
4  *
5  * @param k
6  *         borne supérieure
7  */
8 public void printLowStock (int k)
```

```
1 /**
2  * @param partNumber
3  *         numéro de pièce
4  * @return true si la piece de numero partNumber est disponible
5  *         dans this, false sinon
6  */
7 public boolean isPresent(int partNumber)
8 {
9     return !this.presentAux(partNumber).isEmpty();
10 }
11 /**
12 * @param partNumber
13 *         numéro de pièce
14 * @return itérateur dans lequel le noeud courant repère :
15 *         - l'element trouvé si la réponse est vrai
16 *         - un butoir (ou il aurait dû être) si la reponse est faux
17 */
18 private Iterator<Part> presentAux(int partNumber)
```

```
1 /**
2  * Ajouter newPart a this si aucune pièce de numéro newPart.number
3  * n'est disponible dans this, sans effet sinon.
4  *
5  * @param newPart
6  *         nouvelle pièce
7  */
8 public void addPart (Part newPart)
```

```
1 /**
2  * Diminuer de decrease le nombre d'exemplaires de la pièce de numéro
3  * partNumber chez this (sans effet si absente) et la supprime
4  * si le nombre d'exemplaires devient négatif ou nul.
5  *
6  * @param partNumber
```

```

7      *          numéro de pièce
8      * @param decrease
9      *          nombre de pièces à diminuer
10     */
11     public void diminishStock (int partNumber, int decrease)

```

Rééquilibrage de l'arbre binaire.

La recherche symétrique (fonction *isPresent*) est optimale lorsque l'arbre est de hauteur minimale ($\log_2 n$ si l'arbre possède n nœuds). Pour rééquilibrer l'arbre de temps à autre, on utilise l'attribut d'instance `table = new Part[MAX_SIZE]`.

```

1  /**
2   * Ranger les pièces de l'arbre dans l'ordre croissant dans this.table.
3   */
4  private void arrange()
5  {
6      Iterator<Part> it = this.iterator();
7      arrangeAux(it);
8  }
9
10 /**
11  * @param it
12  *          itérateur d'arbre
13  * @pre Les  $n$  premiers elements du tableau this.table contiennent les
14  *       this.n pieces de plus petits numeros figurant dans l'arbre.
15  * @post Les pieces du sous-arbre "courant" ont ete rangees dans
16  *       this.table, par numeros croissants, a partir de l'indice
17  *       égal à la valeur de this.n au début de la méthode.
18  */
19 private void arrange (Iterator<Part> it)

```

```

1  /**
2   * Reconstruire, à partir de this.table, un arbre de hauteur minimale
3   * this devient cet arbre.
4   */
5  public void reconstruct ()

```

Représentation des arbres binaires

On s'intéresse aux représentations chaînées par références selon 2 techniques : chaînage père et pile de pères.

Chaînage père

Chaque élément d'arbre possède 4 champs : `value`, `father`, `left`, `right`.

```
1 public class BinaryTree<T>
2 {
3     private class Element
4     {
5         public T value;
6         Element father, left, right;
7         public Element()
8         {
9             value = null;
10            father = null;
11            left = null;
12            right = null;
13        }
14        public Element(Element father)
15        {
16            value = null;
17            this.father = father;
18            left = null;
19            right = null;
20        }
21    }
22    public class TreeIterator implements Iterator<T>
23    {
24        private Element currentNode;
25        private TreeIterator() ...
26        ...
27    }
28    private Element root;
29    ...
30 }
```

Exercice 5.

Écrire les constructeurs *BinaryTree* et *TreeIterator*, et les méthodes *goLeft*, *goUp*, *addValue*.

Pile de pères

Dans la section précédente, le lien *father* permet d'atteindre les ancêtres de *currentNode*. Pour économiser de la place, on supprime le champ *father* de chaque élément et on ajoute, à la classe *TreeIterator*, un attribut pile correspondant à une pile d'éléments mémorisant les références des ancêtres de *currentNode*.

Les méthodes de la classe *Stack* sont

```
void push(Element e) // empiler
Element pop()        // dépiler
Element peek()       // sommet
boolean isEmpty()
void clear()
```

```
1 public class BinaryTree<T>
2 {
3     private class Element
4     {
5         public T value;
6         public Element left, right;
7         public Element()
8         {
9             value = null;
10            left = null;
11            right = null;
12        }
13    }
14    public class TreeIterator implements Iterator<T>
15    {
16        private Element currentNode;
17        private Stack<Element> stack;
18        private TreeIterator() ...
19        ...
20    }
21    private Element root;
22    ...
23 }
```

Exercice 6.

Écrire les méthodes *goLeft*, *goUp*, *goRoot* et *remove*.